Flujos binarios con buffer intermedio:

`BufferedInputStream` y `BufferedOutputStream`

La clase `BufferedInputStream` proporciona la capacidad de disponer de un buffer de memoria intermedio entre la aplicación y un flujo de entrada orientado a byte. Esto significa que se puede utilizar entre sistemas de diferente velocidad, pues el buffer permite ajustar velocidades diferentes. Además de esta importante propiedad, son clases que se usan por su eficiencia; es decir, porque realizan la transferencia de flujos de bytes de forma rápida.

`Ejemplo: copia de un archivo binario (imagen, vídeo, audio…)`

BufferedStream_Copiar.java

La clase `BufferedOutputStream` sirve para optimizar algunos de los procesos de escritura de datos del sistema operativo. Aparte de eso, no aporta más funcionalidad en forma de nuevos métodos.

Además, estas clases se utilizan especialmente para trabajar con flujos de red; por ejemplo para descargar recursos por Internet, puesto que la velocidad de descarga no se puede controlar y por tanto las clases con buffer son especialmente adecuadas.

`Ejemplo: descarga de recursos de Internet`

URLCopy.java

URLCopyMain.java

Retroceder en la lectura de un flujo con `BufferedInputStream`

`BufferedInputStream` permite volver atrás en cualquier momento de la lectura o la escritura, al contrario de lo que normalmente se permite. Por tanto, es una clase muy usada en la reproducción de elementos multimedia, pues permite por ejemplo hacer "pause" en una marca y permitir que el multimedia se siga descargando mientras se llena el buffer, pero que la reproducción se reinicie en el punto que se marcó como pausa. Para esto, dispone de métodos adicionales:

- `void mark(int límite).` Marca una posición del flujo. La posición marcada se conserva siempre que no se lean más de `limit` bytes (que correspondería al tamaño del buffer intermedio). En caso de que esto suceda, la marca se pierde. Si bien este método se puede llamar varias veces a lo largo de la vida del flujo, como máximo puede haber una única marca válida. Esta siempre será la correspondiente a la última llamada de este método.
- `void reset().` El flujo retrocede el procesamiento de datos de nuevo hasta la posición marcada. En las lecturas siguientes, se volverán a obtener exactamente los mismos datos que se obtuvieron inmediatamente después de llamar mark.

`Ejemplo: reproducción de un archivo de texto con pausa y retroceso`

BufferedStream_Retroceder.java